Функционал приложения. Валидация. Обработка ошибок.

Содержание

1. В чем проблема? 2
2. Использование перечислений в качестве ошибок. 4
3. Создание объектов-значений. 6
   - 3. 1. Пример объектов значений, когда важна проверка на пустоту: 6
   - 3. 2. Пример объектов значений, когда важна проверка на значение: 9
   - 3. 3. Объекты значений с несколькими полями. 10
   - 3. 4. Объекты значений с проверкой на соответствие формату данных. 13
   - 3. 5. Конвертация между ошибками. 18

1. В чем проблема?

Наши текущие модели выглядат следующим образом:



Заказчик

```rust
pub struct Customer {
    pub(crate) id: Uuid,
    pub(crate) name: String,
    pub(crate) phone_number: String,
    pub(crate) created_at: UtcDateTime,
    pub(crate) updated_at: Option<UtcDateTime>,
    pub(crate) deleted_at: Option<UtcDateTime>,
}
```



Товар

```rust
pub struct Item {
    pub(crate) id: Uuid,
    pub(crate) name: String,
    pub(crate) price: f32,
    pub(crate) created_at: UtcDateTime,
    pub(crate) updated_at: Option<UtcDateTime>,
    pub(crate) deleted_at: Option<UtcDateTime>,
}
```



Заказ

```rust
pub struct Order {
    pub(crate) order_id: Uuid,
    pub(crate) customer_id: Uuid,
    pub(crate) item_id: Uuid,
    pub(crate) quantity: i32,
    pub(crate) created_at: UtcDateTime,
    pub(crate) finished_at: Option<UtcDateTime>,
}
```



Договор

```rust
pub struct Contract {
    pub(crate) id: Uuid,
    pub(crate) customer_id: Uuid,
```

```
    pub(crate) created_at: UtcDateTime,
    pub(crate) finished_at: Option<UtcDateTime>,
}
```

Эти структуры содержат поля, которые ничем не защищены от некорректного использования.

Например номер телефона заказчика:

```
pub struct Customer {
    pub(crate) phone_number: String,
}
```

Здесь он просто выражается как строка, и ничто не мешает записать в это поле некорректный номер телефона (значение, которое не соответствует формату номера телефона, пустое значение, очень длинная строка).

Другой пример - цена товара:

```
pub struct Item {
    pub(crate) price: f32,
}
```

Допустим, что цена товара не может быть отрицательной. В текущей модели ничто не мешает записать в поле `price` отрицательное значение, что является некорректным.

```
pub struct Order {
    pub(crate) quantity: i32,
}
```

В текущей модели ничто не мешает записать в поле `quantity` отрицательное значение, что является некорректным.

Следующий пример кода иллюстрирует, что мы просто можем записать что угодно в эти значения, главное, чтобы тип данных подходил:

```
fn bad_example(mut customer: Customer, mut item: Item, mut order: Order) {

    customer.phone_number = "12345678".to_string(); // номер телефона
некорректный.
    customer.phone_number = "      ".to_string(); // номер телефона пустой.

    item.price = -10.0; // цена отрицательная.
    order.quantity = -5; // количество отрицательное.

}
```

Идентификаторы `Uuid` тоже ничем не защищены от ошибок и некорректного использования.



Это не очень хорошая практика, так как она позволяет записывать некорректные данные в поля структур, что может привести к ошибкам и непредсказуемому поведению приложения.

В реальных приложениях рекомендуется использовать методы для создания и изменения этих структур, которые будут включать валидацию данных, чтобы гарантировать корректность значений.

2. Использование перечислений в качестве ошибок.

Для представления различных ошибок, которые могут возникнуть при работе с заказами, товарами и договорами, можно создать перечисление, где каждый вариант будет соответствовать конкретной ошибке.

Система типов в `Rust` позволяет использовать перечисления (enums) для представления различных вариантов ошибок.

Например, объявим перечень ошибок для некорректных данных в при работе со структурой `Item`:

```
pub enum ItemError {
    InvalidId,                          // некорректный идентификатор
    EmptyId,                            // пустой идентификатор
    EmptyName,                          // пустое название
    NameTooLong,                        // название слишком длинное
    NameTooSmall,                       // название слишком короткое
    InvalidPrice,                       // некорректная цена
    ItemCannotBeUpdatedBeforeCreated, // дата обновления раньше даты создания
    ItemCannotBeDeletedBeforeCreated, // дата удаления раньше даты создания
}
```

Объявление перечисления `ItemError` позволяет явно указать все возможные ошибки, которые могут возникнуть при работе с сущностью `Item`.

Теперь при работе с сущностью `Item` мы можем использовать перечисление `ItemError` для обработки различных ошибок, связанных с некорректными данными.

Например, при создании нового товара, мы можем проверять корректность входных параметров при его создании и возвращать соответствующую ошибку из перечисления `ItemError`, если данные некорректны.

```
    pub fn create(
        id: Uuid,
        name: impl Into<String>,
        price: f32,
        created_at: UtcDateTime,
        updated_at: Option<UtcDateTime>,
        deleted_at: Option<UtcDateTime>,
    ) -> Result<Self, ItemError> {
        if id.is_nil() {
            return Err(ItemError::EmptyId);
        }

        if id.get_variant() == uuid::Variant::RFC4122
            || id.get_version() != Some(uuid::Version::Random)
        {
            return Err(ItemError::InvalidId);
        }
```

```
        let name = name.into();
        let trimmed_name = name.trim();
        if trimmed_name.is_empty() {
            return Err(ItemError::EmptyName);
        }

        if trimmed_name.len() > 255 {
            return Err(ItemError::NameTooLong);
        }

        if trimmed_name.len() < 3 {
            return Err(ItemError::NameTooSmall);
        }

        if price < 0.0 {
            return Err(ItemError::InvalidPrice);
        }

        if let Some(updated_at) = updated_at
            && updated_at < created_at
        {
            return Err(ItemError::ItemCannotBeUpdatedBeforeCreated);
        }

        if let Some(deleted_at) = deleted_at
            && deleted_at < created_at
        {
            return Err(ItemError::ItemCannotBeDeletedBeforeCreated);
        }

        Ok(Self {
            id: id,
            name: trimmed_name.to_string(),
            price: price,
            created_at: created_at,
            updated_at: updated_at,
            deleted_at: deleted_at,
        })
    }
```

Данный код демонстрирует базовую простую валидацию входных параметров при создании нового товара, а также показывает, как можно использовать перечисление `ItemError` для обработки ошибок при создании нового товара.

Также стоит отметить, что метод `create` возвращает результат типа `Result<Self, ItemError>`, что позволяет вызывающему коду явно обрабатывать ошибки и принимать соответствующие меры в случае некорректных данных.

В случае успешного создания нового товара, метод возвращает экземпляр структуры `Item`, который содержит все необходимые поля, включая автоматически сгенерированный идентификатор и временные метки.

3. Создание объектов-значений.

Согласитесь, что, для других структур в нашей системе, проверки могут быть аналогичны. Например, при создании сущности `Order`, `Customer`, `Contract` необходимо также проверять корректность входных данных и возвращать ошибки.

Но здесь мы сталкиваемся с проблемой дублирования кода валидации для каждого поля или набора полей в разных сущностях.

Одним из способов решения этой проблемы является создание типизированных обёрток для критических полей, таких как идентификаторы, названия и цены. Эти обёртки могут инкапсулировать логику валидации и обеспечивать корректность данных на уровне типов, что позволяет избежать дублирования кода и повышает надёжность системы. Такие структуры-обёртки называются ***объектами-значениями***.

Создадим для объектов значений отдельный модуль `value_objects` и будем там определять типизированные обёртки. Модуль будет находится внутри модуля сущностей `entities`.

```
src/
`-- entities/
    |-- mod.rs
    |-- contract.rs
    |-- customer.rs
    |-- item.rs
    |-- order.rs
    `-- value_objects/
        `-- mod.rs
```

3. 1. Пример объектов значений, когда важна проверка на пустоту:

Допустим, мы хотим хранить какие-либо названия, например товаров в нашем приложении. Важно, чтобы эти названия не были пустыми, поэтому мы создадим объект-значение, который будет инкапсулировать эту проверку.

Создать объект-значение `Name`, который будет инкапсулировать проверку на пустоту и обеспечивать корректность названий:

```rust
pub struct Name(String);
```

Для ошибок, связанных с объектом-значением `Name`, можно определить перечисление `NameError`, которое будет содержать все возможные ошибки, такие как пустое имя, слишком длинное имя и т.д.

```rust
pub enum NameError {
    NameIsEmpty,
    NameTooLong,
    NameExceedsMaxLength,
    NameLessThanMinLength,
}
```

Далее определить реализацию структуры `Name`, включая методы для создания нового имени с проверкой на пустоту:

```rust
pub enum FormatOptions {
    Trim,
    FirstLetterToUpper,
}

impl Name {
    #[doc = "Создаёт новое непустое имя"]
    pub fn new(value: String) -> Result<Self, NameError> {
        if value.is_empty() {
            return Err(NameError::NameIsEmpty);
        }
        Ok(Name(value))
    }

    #[doc = "Создаёт новое непустое имя с применением форматирования"]
    pub fn new_with_format(value: String, options: Vec<FormatOptions>) ->
Result<Self, NameError> {
        let mut formatted_value = value.clone();
        for option in options {
            match option {
                FormatOptions::Trim => {
                    formatted_value = Self::trim_value(&formatted_value);
                }
                FormatOptions::FirstLetterToUpper => {
                    formatted_value =
Self::first_letter_to_upper(&formatted_value);
                }
            }
        }
        Name::new(formatted_value)
    }

    #[doc = "Создаёт новое непустое имя с применением форматирования и
проверкой длины"]
    pub fn new_with_format_and_length_check(
        value: String,
        options: Vec<FormatOptions>,
        min_length: usize,
        max_length: usize,
    ) -> Result<Self, NameError> {
        let mut formatted_value = value.clone();
        for option in options {
            match option {
                FormatOptions::Trim => {
                    formatted_value = Self::trim_value(&formatted_value);
                }
                FormatOptions::FirstLetterToUpper => {
                    formatted_value =
Self::first_letter_to_upper(&formatted_value);
                }
            }
        }
        Self::new_with_length_check(formatted_value, min_length, max_length)
```

```
    }

    #[doc = "Создаёт новое непустое имя с проверкой длины"]
    pub fn new_with_length_check(
        value: String,
        min_length: usize,
        max_length: usize,
    ) -> Result<Self, NameError> {
        if value.len() < min_length {
            return Err(NameError::NameLessThanMinLength);
        }
        if value.len() > max_length {
            return Err(NameError::NameExceedsMaxLength);
        }
        Name::new(value)
    }

    fn trim_value(value: &str) -> String {
        value.trim().to_string()
    }

    fn first_letter_to_upper(value: &str) -> String {
        if let Some(first_char) = value.chars().next() {
            return first_char.to_uppercase().collect::<String>() + &value[1..];
        }
        value.to_string()
    }
}
```

Далее, это можно переиспользовать там, где необходимо, например в `Item` (товаре) есть поле названия товара:

```
pub struct Item {
    pub(crate) id: Uuid,
    // раньше был pub(crate) name: String,
    pub(crate) name: Name, // вот где используется тип Name вместо String
}
```

А варианты ошибок с названием товара заменить на `NameError`:

```
pub enum ItemError {
    InvalidId,                        // некорректный идентификатор
    EmptyId,                          // пустой идентификатор
    NameError(NameError),             // ошибка, связанная с названием товара
    InvalidPrice,                     // некорректная цена
    ItemCannotBeUpdatedBeforeCreated, // дата обновления раньше даты создания
    ItemCannotBeDeletedBeforeCreated, // дата удаления раньше даты создания
}
```

Тогда в коде создания и обновления `Item` нужно будет использовать `Name` вместо `String` для названия товара. При этом проверять корректность названия будет уже не нужно, потому что мы уже заранее определили ошибки и проверку на ошибки при создании `Name`:

```rust
impl Item {
    pub fn create(
        id: Uuid,
        name: Name,
        price: f32,
        created_at: UtcDateTime,
        updated_at: Option<UtcDateTime>,
        deleted_at: Option<UtcDateTime>,
    ) -> Result<Self, ItemError> {
        if id.is_nil() {
            return Err(ItemError::EmptyId);
        }

        if id.get_variant() == uuid::Variant::RFC4122
            || id.get_version() != Some(uuid::Version::Random)
        {
            return Err(ItemError::InvalidId);
        }

        if price < 0.0 {
            return Err(ItemError::InvalidPrice);
        }

        if let Some(updated_at) = updated_at
            && updated_at < created_at
        {
            return Err(ItemError::ItemCannotBeUpdatedBeforeCreated);
        }

        if let Some(deleted_at) = deleted_at
            && deleted_at < created_at
        {
            return Err(ItemError::ItemCannotBeDeletedBeforeCreated);
        }

        Ok(Self {
            id: id,
            price: price,
            name: name,
            created_at: created_at,
            updated_at: updated_at,
            deleted_at: deleted_at,
        })
    }
}
```

3. 2. Пример объектов значений, когда важна проверка на значение:

Нам могут так же понадобиться проверки на сами значения, которые должны быть корректными в рамках предметной области.

Например такие понятия как `цена товара` в сущности `Item`, `количество товара` в сущности `Order` должны быть положительными и не могут быть отрицательными числами.

Их тоже можно явно затипизировать в объекты-значения:

```rust
pub struct Price(f64);

pub enum PriceError {
    PriceIsNegative,
}

impl Price {
    pub fn new(value: f64) -> Result<Self, PriceError> {
        if value < 0.0 {
            return Err(PriceError::PriceIsNegative);
        }
        Ok(Price(value))
    }
}
```

```rust
pub struct ItemQuantity(u32);

pub enum ItemQuantityError {
    QuantityIsZero,
}

impl ItemQuantity {
    pub fn new(value: u32) -> Result<Self, ItemQuantityError> {
        if value == 0 {
            return Err(ItemQuantityError::QuantityIsZero);
        }
        Ok(ItemQuantity(value))
    }
}
```

Соответственно, эти структуры можно подключить в сущности:

```rust
pub struct Item {
    pub(crate) id: Uuid,
    pub(crate) name: Name,
    pub(crate) price: Price,
}
```

```rust
pub struct Order {
    pub(crate) id: Uuid,
    pub(crate) item_quantity: ItemQuantity,
}
```

3. 3. Объекты значений с несколькими полями.

Некоторые данные, которые тесно связаны друг с другом, удобно объединять в объекты значений с несколькими полями.

Например есть следующие данные, которые присутствуют во многих сущностях:

```rust
pub struct Item {
    pub(crate) created_at: UtcDateTime,          // Время создания записи
    pub(crate) updated_at: Option<UtcDateTime>, // Время последнего обновления
записи
    pub(crate) deleted_at: Option<UtcDateTime>, // Время удаления записи
}
```

Их можно сгруппировать в объект значения `RecordLifetime`. Потому что логика между ними тесно связана и не зависит от других данных:

```rust
/// Проверка корректности времени обновления записи относительно времени
создания.
if let Some(updated_at) = updated_at
    && updated_at < created_at
{
            return Err(ItemError::ItemCannotBeUpdatedBeforeCreated);
}

/// Проверка корректности времени удаления записи относительно времени
создания.
if let Some(deleted_at) = deleted_at
    && deleted_at < created_at
{
            return Err(ItemError::ItemCannotBeDeletedBeforeCreated);
}
```

```rust
/// Объект значения, представляющий время жизни сущности в системе. Содержит
информацию о времени создания, обновления и удаления записи.
pub struct RecordLifetime {
    pub(crate) created_at: UtcDateTime,
    pub(crate) updated_at: Option<UtcDateTime>,
    pub(crate) deleted_at: Option<UtcDateTime>,
}

pub enum RecordLifetimeError {
    UpdatedBeforeCreated,
    DeletedBeforeCreated,
    DateTimeOutOfRange,
}

impl RecordLifetime {
    pub fn create(
        created_at: UtcDateTime,
        updated_at: Option<UtcDateTime>,
        deleted_at: Option<UtcDateTime>,
    ) -> Result<Self, RecordLifetimeError> {
        let max_valid_datetime = utc_datetime!(2060-01-01 00:00:00);
        let min_valid_datetime = utc_datetime!(2000-01-01 00:00:00);

        if let Some(updated_at) = updated_at
            && updated_at < created_at
        {
```

```
            return Err(RecordLifetimeError::UpdatedBeforeCreated);
        }

        if let Some(deleted_at) = deleted_at
            && deleted_at < created_at
        {
            return Err(RecordLifetimeError::DeletedBeforeCreated);
        }

        if created_at < min_valid_datetime || created_at > max_valid_datetime {
            return Err(RecordLifetimeError::DateTimeOutOfRange);
        }

        if let Some(updated_at) = updated_at {
            if updated_at < min_valid_datetime || updated_at >
max_valid_datetime {
                return Err(RecordLifetimeError::DateTimeOutOfRange);
            }
        }

        if let Some(deleted_at) = deleted_at {
            if deleted_at < min_valid_datetime || deleted_at >
max_valid_datetime {
                return Err(RecordLifetimeError::DateTimeOutOfRange);
            }
        }

        Ok(RecordLifetime {
            created_at,
            updated_at,
            deleted_at,
        })
    }
}
```

Далее можно использовать объект значения `RecordLifetime` для управления временем жизни сущности в системе, обеспечивая корректность данных и соблюдение бизнес-правил.

```
pub struct Item {
    pub(crate) id: Uuid,
    pub(crate) name: Name,
    pub(crate) price: Price,
    pub(crate) record_lifetime: RecordLifetime,
}

impl Item {
    pub fn create(
        id: Uuid,
        name: Name,
        price: Price,
        record_lifetime: RecordLifetime,
    ) -> Result<Self, ItemError> {
        if id.is_nil() {
```

```
            return Err(ItemError::EmptyId);
        }

        if id.get_variant() == uuid::Variant::RFC4122
            || id.get_version() != Some(uuid::Version::Random)
        {
            return Err(ItemError::InvalidId);
        }

        Ok(Self {
            id: id,
            price: price,
            name: name,
            record_lifetime: record_lifetime,
        })
    }
}
```

3. 4. Объекты значений с проверкой на соответствие формату данных.

Некоторые данные, по типу спец. кодов, электронных почт, номеров телефонов, идентификаторов и других формализованных значений, требуют проверки на соответствие определенному формату перед сохранением в системе.

Для этого можно использовать как штатные средства языка программирования, так и специализированные библиотеки для валидации формата данных.

Например, библиотека для работы с идентификаторами `Uuid` позволяет генерировать и проверять корректность UUID, обеспечивая соответствие формату.

```
use uuid::Uuid;

pub struct EntityId(Uuid);

pub enum EntityIdError {
    EmptyId,
    InvalidId,
}

impl EntityId {
    pub fn new() -> Self {
        Self(Uuid::new_v4())
    }

    pub fn create(id: Uuid) -> Result<Self, EntityIdError> {
        if id.is_nil() {
            return Err(EntityIdError::EmptyId);
        }

        if id.get_variant() != uuid::Variant::RFC4122
            || id.get_version() != Some(uuid::Version::Random)
        {
            return Err(EntityIdError::InvalidId);
```

```
        }

        Ok(Self(id))
    }

    pub fn value(&self) -> Uuid {
        self.0
    }
}
```

А его использование:

```
pub struct Item {
    pub(crate) id: ItemId,
    pub(crate) name: Name,
    pub(crate) price: Price,
    pub(crate) record_lifetime: RecordLifetime,
}
```



Обратите внимание, что в конечном итоге, используя объекты-значения для сущностей может привести к тому, что код внутри сущностей станет более простым и лаконичным. Например ниже, нам вообще не нужно делать какие либо проверки и возвращать ошибки при создании сущности `Item`:

```
impl Item {
    pub fn create(id: ItemId, name: Name, price: Price, record_lifetime:
RecordLifetime) -> Self {
        Self {
            id: id,
            price: price,
            name: name,
            record_lifetime: record_lifetime,
        }
    }
}
```

Другим подходом является использование регулярных выражений для проверки соответствия данных определенному формату.

3. 4. 1. Подключение библиотеки regex.

Для работы с регулярными выражениями в Rust используется библиотека `regex`. В файле `Cargo.toml` подключим её отдельной таблицей, как и остальные зависимости проекта:

```
[dependencies.regex]
version = "1"
```

Зависимость уже подключена в проекте. Остальные зависимости сохраняются без изменений.

3. 4. 2. Что такое регулярное выражение?

Регулярное выражение (regular expression, сокращённо regex) - это шаблон, описывающий последовательность символов. Он позволяет искать фрагменты текста или проверять, соответствует ли строка заданному формату.

Для учебного примера определим правило хранения телефона: строка начинается с `7`, после которой идут ровно десять цифр от `0` до `9`. Знак `+`, пробелы, скобки и дефисы не допускаются. Это выбранное правило приложения, а не универсальная проверка всех телефонных номеров.

Шаблон для такого правила: `\A7[0-9]{10}\z`.

- `\A` обозначает начало всей строки.
- `7` обозначает обязательную первую цифру номера.
- `[0-9]` обозначает одну ASCII-цифру. В библиотеке `regex` класс `\d` по умолчанию включает также цифры Unicode.
- `{10}` требует ровно десять повторений предыдущего элемента.
- `\z` обозначает конец всей строки.

Границы строки необходимы, чтобы проверять весь номер, а не находить подходящий фрагмент внутри произвольного текста.

Пробовать создавать и проверять регулярные выражения можно на следующем сайте: https://regex101.com/

3. 4. 3. Использование regex в Rust.

Метод `Regex::new` компилирует шаблон и возвращает `Result<Regex, regex::Error>`. Ошибка здесь означает некорректный шаблон, а не некорректный номер. Метод `is_match` проверяет соответствие строки и возвращает `bool`:

```
use regex::Regex;

fn main() -> Result<(), regex::Error> {
    let phone_regex = Regex::new(r"\A7[0-9]{10}\z")?;

    assert!(phone_regex.is_match("79991234567"));
    assert!(!phone_regex.is_match("+79991234567"));
    assert!(!phone_regex.is_match("89991234567"));
    assert!(!phone_regex.is_match("7 (999) 123-45-67"));
    assert!(!phone_regex.is_match("телефон: 79991234567"));
    assert!(!phone_regex.is_match("79991234567\n"));

    Ok(())
}
```

Запись `r"..."` является строковым литералом без обработки escape-последовательностей Rust. Благодаря этому обратные слеши шаблона не нужно удваивать.

Компиляция шаблона требует ресурсов, поэтому один и тот же regex не следует создавать заново при каждой проверке. Для многократного использования подходит `std::sync::LazyLock`, доступный начиная с Rust 1.80: шаблон компилируется при первом обращении, а затем используется повторно.

3. 4. 4. Объект-значение номера телефона и сущность Customer.

По аналогии с `Name` определим объект-значение `PhoneNumber`: структуру с одним приватным полем, перечисление ошибок и проверяющий конструктор. В проекте они размещены в файле `src/entities/value_objects/phone_number.rs`:

```
use regex::Regex;
use std::sync::LazyLock;

static PHONE_REGEX: LazyLock<Regex> = LazyLock::new(|| {
    Regex::new(r"\A7[0-9]{10}\z").expect("Phone regex must be valid")
});

pub struct PhoneNumber(String);

pub enum PhoneNumberError {
    EmptyNumber,
    InvalidFormat,
}

impl PhoneNumber {
    pub fn new(value: impl Into<String>) -> Result<Self, PhoneNumberError> {
        let value = value.into();

        if value.trim().is_empty() {
            return Err(PhoneNumberError::EmptyNumber);
        }

        if !PHONE_REGEX.is_match(&value) {
            return Err(PhoneNumberError::InvalidFormat);
        }

        Ok(Self(value))
    }

    pub fn value(&self) -> &str {
        &self.0
    }
}
```

В файле `src/entities/value_objects/mod.rs` объявим подмодуль и экспортируем его типы рядом с остальными объектами-значениями:

```
mod phone_number;

pub use phone_number::*;
```

Модуль `entities` уже экспортирует содержимое `value_objects`, поэтому `PhoneNumber` и `PhoneNumberError` доступны через `crate::entities`.

Поле внутри `PhoneNumber` приватное: вызывающий код создаёт значение через проверяющий конструктор. Пробельная строка возвращает `EmptyNumber`, а непустая строка с лишними пробелами или другим форматом - `InvalidFormat`. Метод `value` предоставляет только чтение номера без копирования строки.

Здесь `expect` применяется к постоянному шаблону разработчика. Ошибка шаблона является ошибкой программы; некорректные пользовательские номера не вызывают панику, а возвращаются через `PhoneNumberError`.

В файле `src/entities/customer.rs` поле `phone_number` теперь имеет тип `PhoneNumber` вместо `String`. Оба конструктора получают уже проверенный номер, а перечисление ошибок включает `PhoneNumberError`. Остальные поля используют существующие объекты-значения:

```
use crate::entities::{
    EntityId, EntityIdError, Name, NameError, PhoneNumber, PhoneNumberError,
    RecordLifetime, RecordLifetimeError,
};
use time::UtcDateTime;

pub struct Customer {
    pub(crate) id: EntityId,
    pub(crate) name: Name,
    pub(crate) phone_number: PhoneNumber,
    pub(crate) record_lifetime: RecordLifetime,
}

pub enum CustomerError {
    IdError(EntityIdError),
    NameError(NameError),
    PhoneNumberError(PhoneNumberError),
    RecordLifetimeError(RecordLifetimeError),
}

impl Customer {
    pub fn create_new(name: Name, phone_number: PhoneNumber) -> Result<Self,
CustomerError> {
        let record_lifetime = RecordLifetime::create(UtcDateTime::now(), None,
None)
            .map_err(CustomerError::RecordLifetimeError)?;
        Ok(Self::create(
            EntityId::new(),
            name,
            phone_number,
            record_lifetime,
        ))
    }

    pub fn create(
        id: EntityId,
        name: Name,
        phone_number: PhoneNumber,
        record_lifetime: RecordLifetime,
    ) -> Self {
        Self {
            id,
            name,
            phone_number,
            record_lifetime,
```

```
        }
    }
}
```

Перед созданием заказчика вызывающий код создаёт проверенный номер через `PhoneNumber::new`. Например, вспомогательная функция принимает строку, оборачивает ошибку в `CustomerError` с помощью `map_err` и передаёт объект-значение в `Customer::create_new`:

```
use crate::entities::{Customer, CustomerError, Name, PhoneNumber};

fn create_customer(
    name: Name,
    phone_number: impl Into<String>,
) -> Result<Customer, CustomerError> {
    let phone_number = PhoneNumber::new(phone_number)
        .map_err(CustomerError::PhoneNumberError)?;

    Customer::create_new(name, phone_number)
}
```

Строка `79991234567` успешно проходит проверку. Строки `+79991234567`, `89991234567`, номера с пробелами, буквами или неверной длиной возвращают `InvalidFormat`. Пустая или состоящая только из пробельных символов строка возвращает `EmptyNumber`. Конструктор не удаляет пробелы и не преобразует номер автоматически.

Конструкторы `Customer` повторно не проверяют телефон: они получают уже проверенный объект-значение. `Customer::create` возвращает `Self`, а `Customer::create_new` возвращает `Result`, поскольку создаёт и проверяет `RecordLifetime`. Перечисление `CustomerError` объединяет ошибки на этапе подготовки данных. Для чтения сохранённого номера используется `customer.phone_number.value()`.



Регулярное выражение проверяет только выбранный формат. Оно не подтверждает, что номер выделен оператором, существует или принадлежит заказчику.

Описание API, синтаксиса шаблонов и повторного использования regex: официальная документация библиотеки regex.

3. 5. Конвертация между ошибками.

Сейчас у нас определены различные типы ошибок для каждого объекта значения:

```
pub enum EntityIdError {
    EmptyId,
    InvalidId,
}
```

```
pub enum ItemQuantityError {
    QuantityIsZero,
}
```

```rust
pub enum NameError {
    NameIsEmpty,
    NameTooLong,
    NameExceedsMaxLength,
    NameLessThanMinLength,
}
```

```rust
pub enum PhoneNumberError {
    EmptyNumber,
    InvalidFormat,
}
```

И возможно еще другие.

Нам может потребоваться конвертировать ошибки из одного типа в другой, чтобы объединить их в единое перечисление ошибок на уровне бизнес-логики. Например, можно создать перечисление `ItemError`, которое будет включать все возможные ошибки, связанные с созданием и валидацией товара:

```rust
pub enum ItemError {
    EntityIdError(EntityIdError),
    ItemQuantityError(ItemQuantityError),
    NameError(NameError),
    PhoneNumberError(PhoneNumberError),
}
```

Чтобы удобно конвертировать ошибки из отдельных типов в `ItemError`, можно реализовать трейт `From` для каждого типа ошибки:

```rust
impl From<EntityIdError> for ItemError {
    fn from(err: EntityIdError) -> Self {
        ItemError::EntityIdError(err)
    }
}

impl From<ItemQuantityError> for ItemError {
    fn from(err: ItemQuantityError) -> Self {
        ItemError::ItemQuantityError(err)
    }
}

impl From<NameError> for ItemError {
    fn from(err: NameError) -> Self {
        ItemError::NameError(err)
    }
}

impl From<PhoneNumberError> for ItemError {
    fn from(err: PhoneNumberError) -> Self {
        ItemError::PhoneNumberError(err)
    }
}
```

Далее это можно использовать следующим образом:

```
impl Item {
    pub fn create(
        id: Uuid,
        name: String,
        price: f64,
        created_at: UtcDateTime,
        updated_at: Option<UtcDateTime>,
        deleted_at: Option<UtcDateTime>,
    ) -> Result<Self, ItemError> {
        let id: EntityId = match EntityId::create(id).map_err(|e| e.into()) {
            Ok(id) => id,
            Err(e) => return Err(e),
        };

        let name =
            match Name::new_with_format(name, vec!
[FormatOptions::Trim]).map_err(|e| e.into()) {
                Ok(name) => name,
                Err(e) => return Err(e),
            };

        let price = match Price::new(price).map_err(|e| e.into()) {
            Ok(price) => price,
            Err(e) => return Err(e),
        };

        let record_lifetime = match RecordLifetime::create(created_at,
updated_at, deleted_at)
            .map_err(|e| e.into())
        {
            Ok(record_lifetime) => record_lifetime,
            Err(e) => return Err(e),
        };

        Ok(Self {
            id: ItemId(id),
            price: price,
            name: name,
            record_lifetime: record_lifetime,
        })
    }
}
```

Здесь, при помощи `into()` мы конвертируем ошибки из отдельных типов в `ItemError`, под капотом неявно используется реализация трейта `From` для каждого типа ошибки.

Мы можем упростить этот код ещё больше, используя оператор `?` вместо явного сопоставления с `match` и возврата ошибки. Например:

```
impl Item {
    pub fn create(
        id: Uuid,
```

```
        name: String,
        price: f64,
        created_at: UtcDateTime,
        updated_at: Option<UtcDateTime>,
        deleted_at: Option<UtcDateTime>,
    ) -> Result<Self, ItemError> {
        let id: EntityId = EntityId::create(id)?.into();
        let name = Name::new_with_format(name, vec!
[FormatOptions::Trim])?.into();
        let price = Price::new(price)?.into();
        let record_lifetime = RecordLifetime::create(created_at, updated_at,
deleted_at)?.into();

        Ok(Self {
            id: ItemId(id),
            price: price,
            name: name,
            record_lifetime: record_lifetime,
        })
    }
}
```

Это более компактный и читаемый способ обработки ошибок, так как оператор `?` автоматически конвертирует ошибки в нужный тип с помощью трейта `From` и сразу же выполняет возврат ошибки из функции, если она произошла.

Задача 1.
Используя вышеперечисленные советы по обработке ошибок, добавьте в ваш проект объекты значения и ошибки для всех ваших сущностей.